NTU-114-1-ADL-HW1

Chia-Chun Liang
Data Science Degree Program
National Taiwan University
Student ID: d14948004
Email: d14948004@ntu.edu.tw

September 2025

Task Description and Overview

The objective of this assignment is to identify the correct answer span from long paragraphs in response to a given user question. The task is designed as a pipeline consisting of two stages:

1. **Paragraph selection** (Stage 1): From four candidate paragraphs, select the one that is most likely to contain the correct answer.
2. **Answer span extraction** (Stage 2): From the chosen paragraph, identify the exact answer span.

which is also shown in Figure 1.

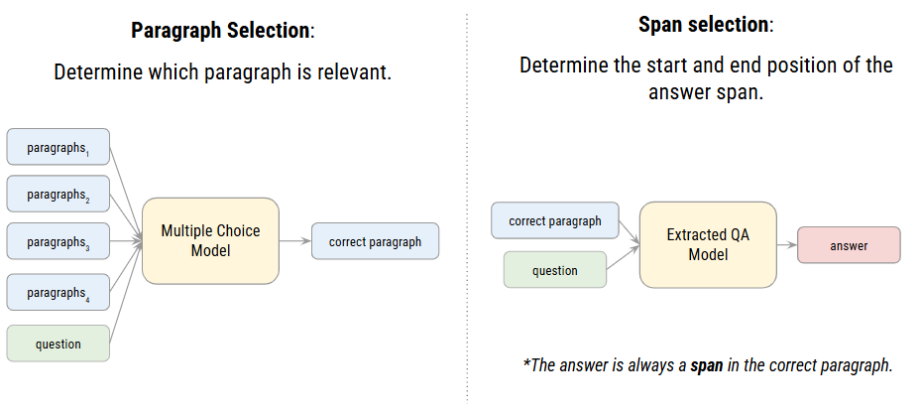


Figure 1: An overview of the whole task process.

I learned that the main bottleneck of this task lies in the extraction of the **answer span** from the validation accuracy, as shown in this report.

Report - Q1: Data Processing (2%)

Tokenizer (1%)

For this task, I used the tokenizer provided with the original pretrained model. The reason is that the model was trained with its corresponding tokenizer. Note that tokenization plays a crucial role in aligning text with the model's internal representation. If a different tokenizer was used, the token IDs might not match the embedding space, which would lead to inconsistent results. In other words, the tokenizer ensures that the input text is segmented into tokens in exactly the same way as the model was trained on, thus maintaining compatibility during fine-tuning.

As shown in Figure 2, the tokenizer processes the input text into subword units, ensuring that rare or unknown words are still decomposed into meaningful components.

```
>>> print(tokenizer)
BertTokenizerFast(name_or_path='hw1_mc_roberta_wwm_ext_e3', vocab_size=21128, model_max_length=1000000000000000019884624838656,
is_fast=True, padding_side='right', truncation_side='right', special_tokens={'unk_token': '[UNK]', 'sep_token': '[SEP]', 'pad_to
ken': '[PAD]', 'cls_token': '[CLS]', 'mask_token': '[MASK]'}, clean_up_tokenization_spaces=False, added_tokens_decoder={
        0: AddedToken("[PAD]", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
        100: AddedToken("[UNK]", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
        101: AddedToken("[CLS]", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
        102: AddedToken("[SEP]", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
        103: AddedToken("[MASK]", rstrip=False, lstrip=False, single_word=False, normalized=False, special=True),
}
)
```

Figure 2: An illustration of the tokenization process.

Answer Span (1%)

During answer selection, the model outputs a score (logit) for each token being the potential start or end of an answer. Since carefully examizing the validation outcomes, I think there are some situations to revise:

- Extra spaces
- Lower cases and upper cases of English words
- Some *top-k* candidates
- Empty string (“”)
- Too-long outputs
- Unpaired brackets.

After counseling with AI tools, I took the top-k (k=10) positions for start and end separately, then form candidate spans by pairing them, while filtering out invalid spans (such as when the end comes before the start, or the span length exceeds a maximum). Each candidate is scored, commonly by adding its start and end logits. The span with the highest score is chosen as the final answer.

Report - Q2: Modeling with BERTs and their variants (4%)

Describe (2%)

- Your model.
- The performance of your model.
- The loss function you used.
- The optimization algorithm (e.g. Adam), learning rate and batch size.

Try another type of pre-trained LMs and describe (2%)

- The new model.
- The performance of this model.
- The difference between pre-trained LMs (architecture, pretraining loss, etc.)
  - For example, BERT → xlnet, or BERT → BERT-wwm-ext. You can find these models in the huggingface's Model Hub.

The above requirements are contained in the following paragraph:

Model & Performance

I tried models including `google-bert/bert-base-chinese`, `hfl/chinese-bert-wwm`, `hfl/chinese-macbert-base`, and so on. (See Appendix). I followed the codes (Stage 1 and Stage 2) with the instruction. I struggled with the parameters and the models. In the end, after discussing with classmates, I chose `hfl/chinese-roberta-wwm-ext` and `hfl/chinese-lert-base` [Cui+22] because they would be good enough with stable performance.

It's worth to mention that *large*-variant may overfitting quite seriously in my experiments. Also, the random seed affects the performance quite a lot.

The performance I used with validation performance is shown in Table 1:

Model (Stage 1/2)	**Validation Accuracy (Exact macth)**
google-bert/bert-base-chinese (Stage 1)	0.94
hfl/chinese-roberta-wwm-ext (Stage 1)	0.9634
google-bert/bert-base-chinese (Stage 2)	0.6
hfl/chinese-roberta-wwm-ext (Stage 2)	0.7930
hfl/chinese-lert-base (Stage 2)	0.7989

Table 1: The Validation accuracy in Stage 1 and 2 with the listing models. The accuracy is counted correct if it exactly matches the label.

Loss function

I used the default loss provided by huggingface which is *cross-Entropy Loss*. This loss measures the dissimilarity between the true label distribution y and the predicted probability distribution $\hat{y}$. For a classification task with N samples and C classes, it is defined as:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c} \log \hat{y}_{i,c}$$

In Stage 2, Claude suggested another option,*focal loss*, which has its root in object detection, which also addresses a problem similar to searching for a tiny span within a large amount of information.

Unfortunately, when I applied `hfl--chinese-roberta-wwm-ext` with the parameters `focal_alpha 1.0` and `focal_gamma 2.0`, thought the loss decreased more smoothly (shown in Fig 3), the final *eval_exact* was about 81% (shown in Fig 4). This did not outperform the result obtained with cross-entropy (approximately 83%, shown in Fig 8). Therefore, I did not adopt focal loss at this stage (though it cannot exclude the possiblity that my parameter choice for focal loss was not optimal).

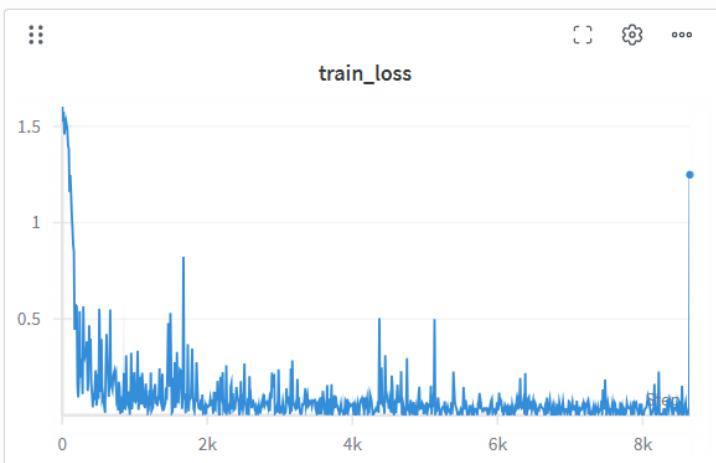


Figure 3: An illustration of the training loss of pretrained `hfl/chinese-roberta-wwm-ext` with **focal loss** with the parameters `focal_alpha 1.0` and `focal_gamma 2.0` for 3 epochs. The final loss is around 0.027.

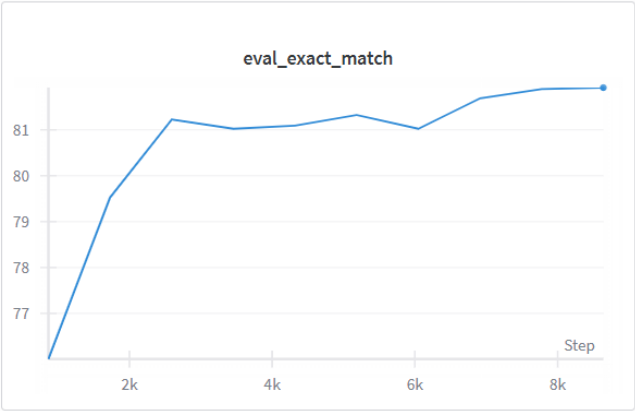


Figure 4: An illustration of the evaluation (Exact Match) of pre-trained `hfl/chinese-roberta-wwm-ext` with **focal loss** with the parameters `focal_alpha 1.0` for 3 epochs. The best exact match metric is around 81.92 (%).

Parameters and optimization

Parameter	**Stage 1**	**Stage 2**
Max sequence length	512	512
Learning rate	1×10^{-5}	1×10^{-5}
Seed	42	42
Weight decay	–	1×10^{-2}
Warmup steps	–	1000
Batch size (train)	1	1

Table 2: Training hyperparameters for Stage 1 (multiple-choice) and Stage 2 (question answering).

Parameters To avoid ‘overfitting‘, and with the hope to achieve better performance, I reduced the learning rate from the assignment setting of 3×10^{-5} to 1×10^{-5}. I did not lower it too much for fear of excessively small learning rate to slow down learning.

Second, I adjusted the maximum sequence length. Once I considered extending it to even 2048, but the outputs became much longer than the correct answers. After the discussion with some classmates, I've been noted that BERT-based models often degrade significantly beyond 512 tokens.

Third, I experimented with different random seeds (e.g., 2025, 128) and found that 42 produced the most stable results. Finally, since **Stage 2** performed worse than **Stage 1** and was more prone to overfitting, I applied *warmup steps*, a progressively increasing of learning rate for stable training, and *weight*

decay, which is one of regularization to reduce overfitting as the following paragraph.

Optimization In both stages, I followed the recommended material which also applied the **AdamW** optimizer, with slightly different because:

- For Stage 1 (Multiple Choice), I set the weight decay to 0, which is equivalent to using **Adam** without weight decay.
- For Stage 2 (Question Answering), I enabled weight decay to reduce overfitting and improve generalization.

Report - Q3: Curves (1%)

- Plot the learning curve of your span selection (extractive QA) model. You can plot both the training and validation set or only one set. Please make sure there are at least 5 data points in each curve.
  - Learning curve of the loss value (0.5%)
  - Learning curve of the Exact Match metric value (0.5%)

I will discuss the above requirements in the following paragraph: Generally, the shape of Training loss is like Figure 5:

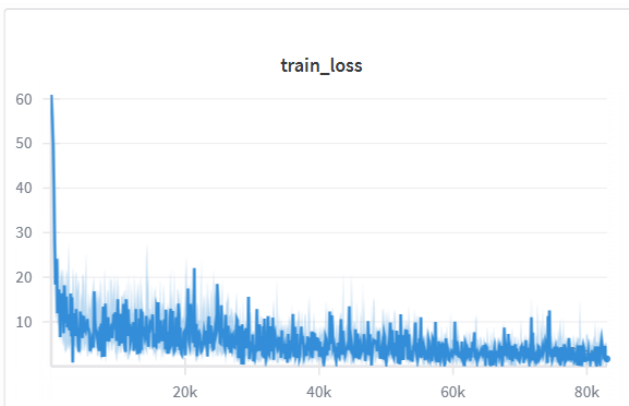


Figure 5: An illustration of the training loss of pretrained `hfl/chinese-lert-base` for 3 epochs. The final loss is around 1.686.

And the evaluation metric is calculated by Exact-Match provided by huggingface every 0.5 epochs, as shown in Figure 6:

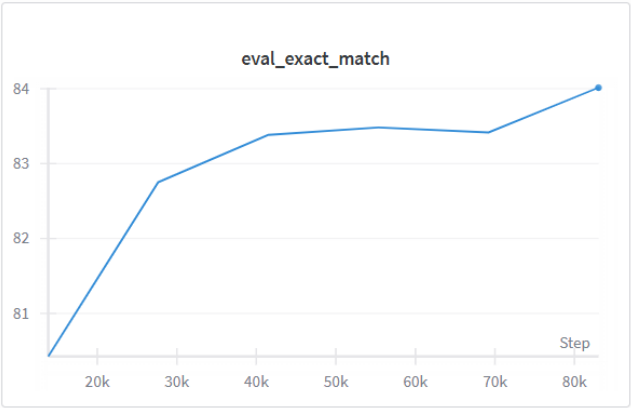


Figure 6: An illustration of the evaluation (Exact Match) of pretrained `hfl/chinese-lert-base` for 3 epochs. The best exact match metric is around 84.014 (%).

`hfl/chinese-roberta-wwm-ext` is also shown in Fig 7 and Fig 8:

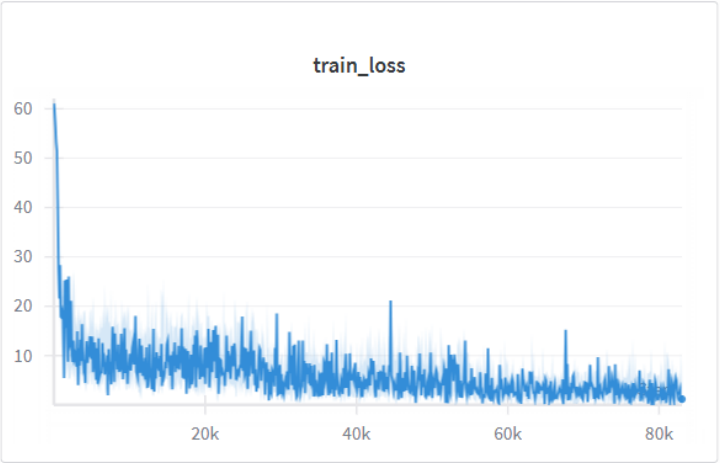


Figure 7: An illustration of the training loss of pretrained `hfl/chinese-roberta-wwm-ext` for 3 epochs. The final loss is around 1.176.

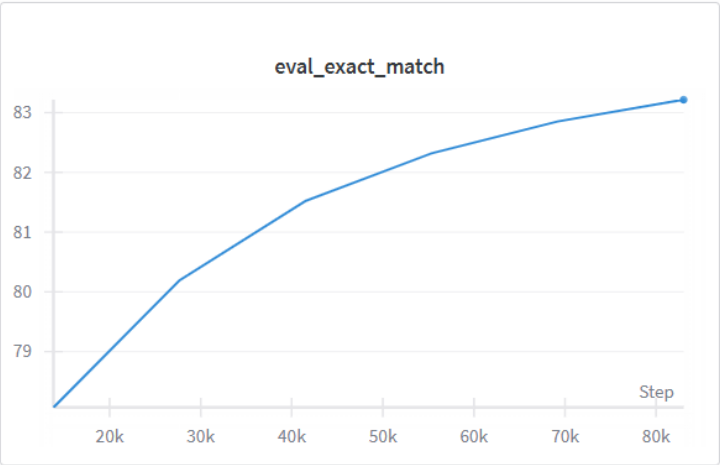


Figure 8: An illustration of the evaluation (Exact Match) of pretrained `hfl/chinese-roberta-wwm-ext` for 3 epochs. The best exact match metric is around 83.217 (%).

It's quite interesting that the priority of the final loss (*hfl/chinese-roberta-wwm-ext* is better) doesn't directly lead the priority of the best exact match metric (*hfl/chinese-lert-base* is better).

Apart from that, quite surprisingly, the training loss will suddenly increase sharply after around one epoch during the *large*-invariant training, even exceeding the original loss function, as shown in Figure 9:

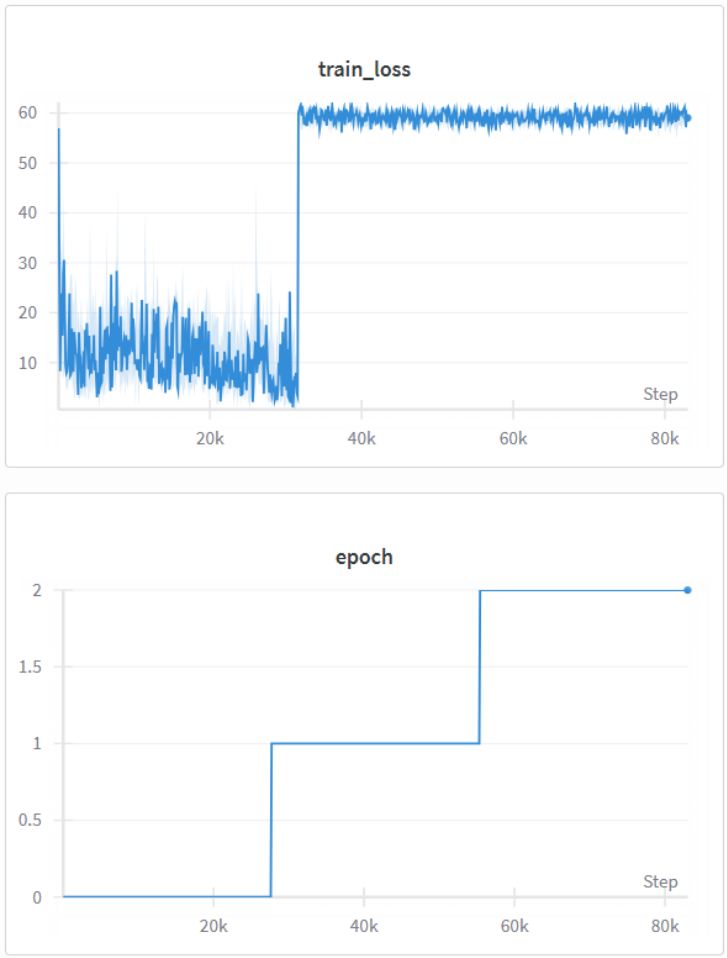
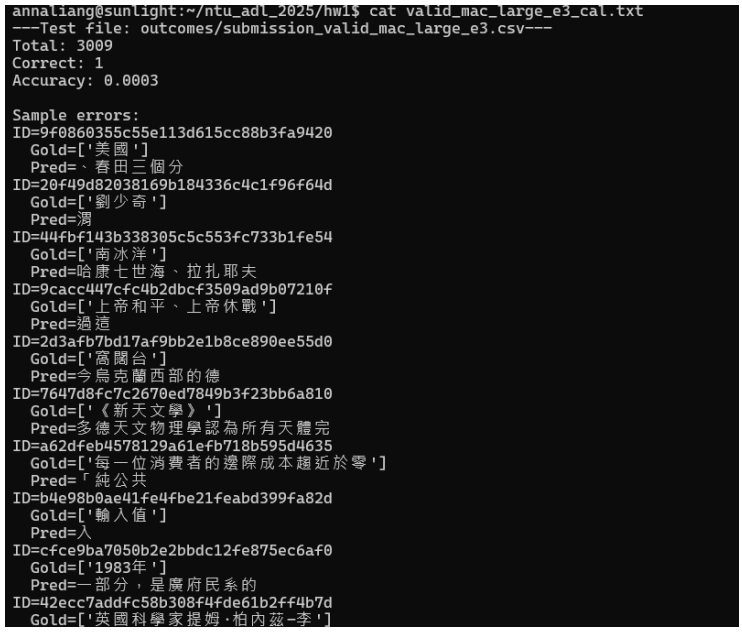


Figure 9: An illustration of the training loss of pretrained `hfl/chinese-macbert-large` for 3 epochs (default parameters including learning rate) .

And the performance is quite strange as shown in Figure 10:



```
annaliang@sunlight:~/ntu_adl_2025/hw1$ cat valid_mac_large_e3_cal.txt
---Test file: outcomes/submission_valid_mac_large_e3.csv---
Total: 3009
Correct: 1
Accuracy: 0.0003

Sample errors:
ID=9f0860355c55e113d615cc88b3fa9420
  Gold=['美國']
  Pred=、春田三個分
ID=20f49d82038169b184336c4c1f96f64d
  Gold=['劉少奇']
  Pred=潤
ID=44fbf143b338305c5c553fc733b1fe54
  Gold=['南冰洋']
  Pred=哈康七世海、拉扎耶夫
ID=9cacc447cfc4b2dbcf3509ad9b07210f
  Gold=['上帝和平、上帝休戰']
  Pred=過這
ID=2d3afb7bd17af9bb2e1b8ce890ee55d0
  Gold=['窩闊台']
  Pred=今烏克蘭西部的德
ID=7647d8fc7c2670ed7849b3f23bb6a810
  Gold=['《新天文學》']
  Pred=多德天文物理學認為所有天體完
ID=a62dfeb4578129a61efb718b595d4635
  Gold=['每一位消費者的邊際成本趨近於零']
  Pred=「純公共
ID=b4e98b0ae41fe4fbe21feabd399fa82d
  Gold=['輸入值']
  Pred=入
ID=cfce9ba7050b2e2bbdc12fe875ec6af0
  Gold=['1983年']
  Pred=一部分，是廣府民系的
ID=42ecc7addfc58b308f4fde61b2ff4b7d
  Gold=['英國科學家提姆·柏內茲-李']
```

Figure 10: An illustration of the abnormal output of pretrained `hfl/chinese-macbert-large` for 3 epochs (default parameters including learning rate) .

After that, I discovered that everything was by default except for the model is large and the `seed` configuration was set to `None`. Maybe this would be the cause to the training uncontrolled instability observed around the 30k step mark. And with addition to a relatively aggressive learning rate of 3e-5, the randomness created a quite changing conditions that triggered the dramatic loss spike as shown in Figure 9. It also highlighted the critical importance of setting deterministic seeds for stable and reproducible fine-tuning of a LM.

Report - Q4: Pre-trained vs Not Pre-trained (2%)

- Plot the learning curve of your span selection (extractive QA) model. You can plot both the training and validation set or only one set. Please make sure there are at least 5 data points in each curve.
  - Learning curve of the loss value (0.5%)
  - Learning curve of the Exact Match metric value (0.5%)

I followed the QA training script described above, but removed the `--model_name_or_path` argument and replaced it with:

```
--config_name hfl/chinese-lert-base
--tokenizer_name hfl/chinese-lert-base
```

The model architecture remains the same as `hfl/chinese-lert-base`. The training loss is shown in Fig 11 and the exact match metric is as Fig 12:

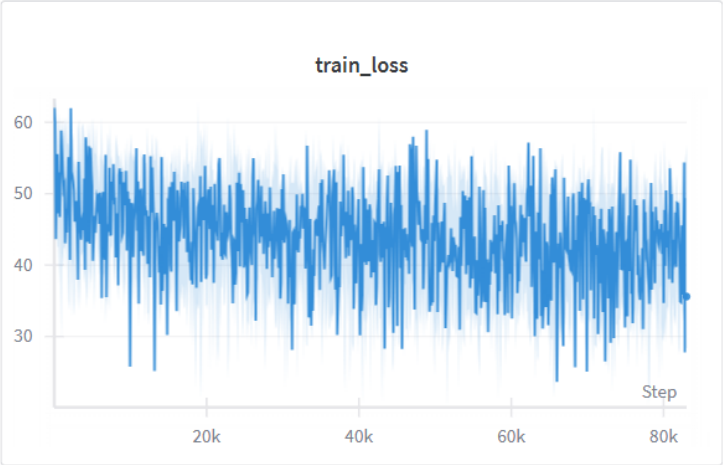


Figure 11: An illustration of the training loss of pretrained `hfl/chinese-roberta-wwm-ext` for 3 epochs. The final loss is around 35.591.

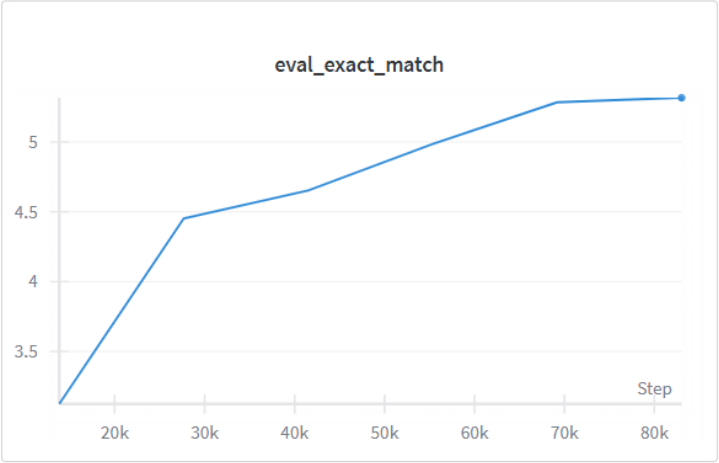


Figure 12: An illustration of the evaluation (Exact Match) of pretrained `hfl/chinese-roberta-wwm-ext` for 3 epochs. The best exact match metric is around 5.317 (%).

It's quite interesting that even the loss from non-pretrained model (around 35.591 (%)) seems to be better than the randomly overfitting (around 60).

Report - Q5: Bonus (2%)

- Instead of the paragraph selection + span selection pipeline approach, train an end-to-end transformer-based model and describe
  - Your model.
  - The performance of your model.
  - The loss function you used.
  - The optimization algorithm (e.g. Adam), learning rate and batch size.
  - Hint: Try models that can handle long input (e.g., models that have a larger context windows).

I used a `BigBird` transformer model Lowin/chinese-bigbird-base-4096 for End-to-End. This model can handle maximum tokens up to 4096 ($8 \times$ to the ones that the Bert series can handle).

For each question q, the model processes four input pairs: $\{(q, c_1), (q, c_2), (q, c_3), (q, c_4)\}$. Each pair is tokenized using a custom JiebaTokenizer provided by the author for Chinese text segmentation.

0.1 Loss Function

The Loss function is the default cross-entropy loss

0.2 Parameter Configuration

- **Optimizer**: AdamW with weight decay = 0.01
- **Learning Rate**: 2e-5 (lower rate for Chinese models)
- **Batch Size**: 2 per device with gradient accumulation steps = 4 (effective batch size = 8)
- **Maximum Sequence Length**: 4096 tokens
- **Training Epochs**: 3
- **Learning Rate Scheduler**: Linear decay with warmup

0.3 Model Performance

The training loss and the evaluation (Exact Match) can be shown in the Figure 13 and Figure 14:

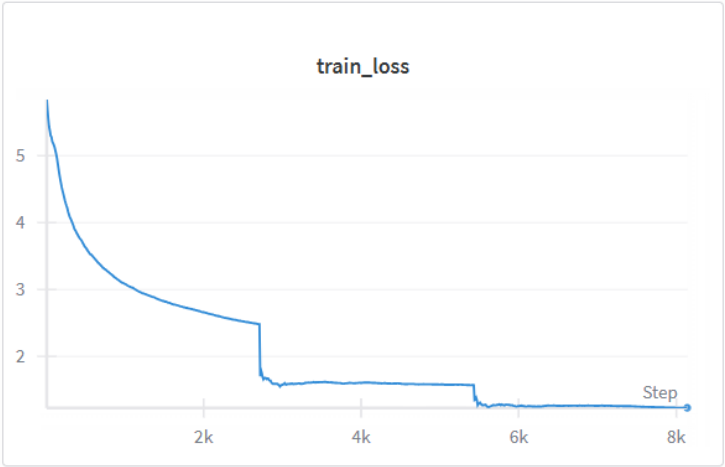


Figure 13: An illustration of the training loss of pretrained `Lowin/chinese-bigbird-base-4096` for 3 epochs. The final loss is around 0.04.

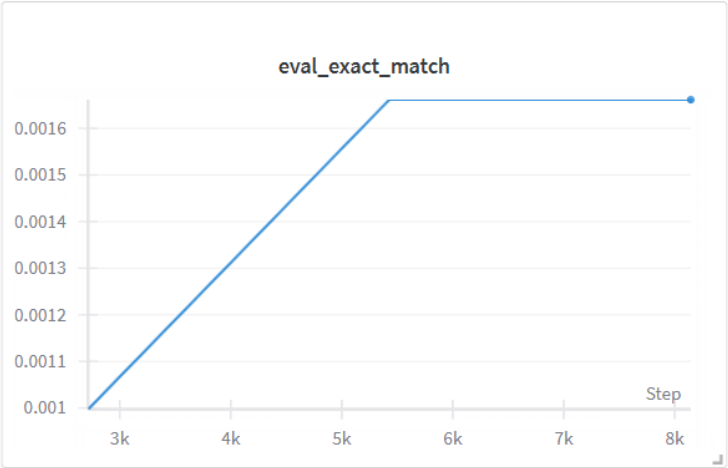


Figure 14: An illustration of the evaluation (Exact Match) of pretrained `Lowin/chinese-bigbird-base-4096` for 3 epochs. The best exact match metric is around 16.61 (%).

At the end the test score on kaggle leaderboard was 14.50%. It's quite a pity that the overall performance turned out to be unsatisfactory. It would remain an opening task for future developments.

```
id,answer
5e7a923dd6e4ccb8730eb95230e0c908,年
a2e9cd802197b8f8dfbe235e2761f9ed,列寧
c7c8a85b3f0006d44d86510a22193620,年
7f4f68726faed6b987e348340a9e6a61,石油收入
89908ef5182021a9aec1472c5bbcbd8c,中國地質大學逸夫博物館
14f4fbc7e40d017836f414e71acead71,蜀漢
4911c04597a5e2d7eb539efa00c053f8,中國改革開放
755ebc079613e214f1abd5d52a6e8d32,笛卡
38e45c1c2c6e7a96080bd753c7fadb43,洲太平洋經合作會議
d52156507573c4742d4e75a887e5b5d7,阿德里安堡役
196459cd4f5ef8c3131171430d4eeaca,萬公里
f5594e2e303b34fb1620da127d930d9c,有體
a5ba4f33e0a83a9ecba9b0f3adfd7b14,年東德政府布第二版法時
63ea80494c552fecb4b54eb64f226686,第二次世界大
ba806d826731e8dc38c8e8f1300b6726,麻術
9fe42ca2d1efeaa8e1fd45120612cc8e,謝芝芬
ddbea2ca7dda8989670a5ff773eb243a,生育率
d82f4c4513687055ce042071780452e4,隋唐時代
3d5d21134f95a76609c8441eb73813c4,蕭查理博士
4c2f00fb03ad43b8209325c21201905a,可克森
39ab6557763c3f915179675dedf63b30,年
adb2675ed651e3f67ab671f82cc264fb,國家中心城市
b8664bcbea5402f9fa5885b9ed231f12,北美
e2d2afe821fc59ddc9227d64e35c7c32,命令列介面
e84fd3b41339ec4558004d66ebda27d0,陸地
f7dba33d747a99b0429e08b4c1ac3363,自然科學
0ab024ea410b8dcf969f868e4c114fe7,西哀士
37bc5ae3cd3acd6788727c388c397b1d,中華民國
d2a6608bafac0e882e002bdddc63db64,索塔程車
```

Figure 15: An illustration of the real output after fine-tuning.

Acknowledgement

I used ChatGPT and Claude for parameter adjustment, code crafting and report polishing.

I've discussed with classmates with model selecting and knowing the meaning of the setting of random seed.

Appendix

Ablation Experiment Results and Analysis

I first trained the baseline model `google-bert/bert-base-chinese` for three epochs, following the codes (Stage 1 and Stage 2). The validation accuracy for Stage 1 (paragraph selection) reached 0.9471, while the validation accuracy for Stage 2 (answer span extraction, evaluated only on correctly selected paragraphs) achieved 0.6766.

To compare the performance of different pre-trained LMs, I conducted experiments using several models from HuggingFace. A summary of the results is presented in Table 3 (random seed set '2025'):

Model	Correct(Overlapping)	**Validation Accuracy (Overlapp**
google-bert/bert-base-chinese (3e)	2036/3009	0.6766
google-bert/bert-base-chinese (5e)	1986/3009	0.6600
algolet/bert-large-chinese (3e)	1968/3009	0.6540
google-bert/bert-base-chinese (lr*: $2e^{-5}$) (3e)	2060/3009	0.6846
hfl/chinese-bert-wwm (3e)	2020/3009	0.6713
hfl/chinese-roberta-wwm-ext (3e)	2058/3009	0.6839
hfl/chinese-macbert-base (3e)	2076/3009	0.6899
hfl/chinese-macbert-large (3e)	2088/3009	0.6939
hfl/chinese-lert-base (3e)	2113/3009	0.7022
hfl/chinese-lert-large (3e)	0/3009	0.0000
hfl/chinese-electra-base-discriminator (3e)	2140/3009	0.7112

Table 3: Comparison of different pre-trained language models for answer span extraction (Stage 2). Correct items means for the number of exactly correction in 3009 validation dataset. Validation accuracy where a prediction is counted correct if it is overlapping any part of the golden answers, for different hyperparameter settings (without post-preprocess). Lr* means the parameter of learning rate.

Starting with the BERT-base models, I observe that *google-bert/bert-base-chinese* performs best when the learning rate is set to 2×10^{-5} (EM = 0.6846), outperforming both the 3-epoch (0.6766) and 5-epoch (0.6600) settings. This suggests that the choice of learning rate has a greater impact on performance than simply increasing the number of epochs.

Among the other *BERT*-family models, *hfl/chinese-bert-wwm* and *hfl/chinese-roberta-wwm-ext* achieve EM scores of 0.6713 and 0.6839, respectively, comparable to *BERT*-base. *hfl/chinese-macbert-base* further improves performance to 0.6966, indicating that *MacBERT*'s improvements in semantic representation contribute positively to this task.

The best-performing model overall is *hfl/chinese-electra-base-discriminator*, which achieves an accuracy of 0.7112. However, I do not exclude the potential of other strong models. Meanwhile, recent work [Cui+22] suggests that theoret-

ically hfl/chinese-lert-base and hfl/chinese-lert-large may be stronger, indicating further room for improvement beyond the current candidates. For further comparison, I select the *MacBERT*, *electra* and lert-series to continue our experiments.

Model / Setting	**Validation Accuracy (Overlapping)**
`MacBERT (hfl/chinese-macbert-base)`	
3 epochs (default)	0.6899
+ Learning rate = $2e^{-5}$ (3e)	0.7205
+ Random seed = 2025 $\rightarrow$ 128 (3e)	0.6956
+ Weight decay = 0.01 (3e)	0.6996
`MacBERT (hfl/chinese-macbert-large)`	
3 epochs (default)	0.6939
+ Learning rate = $2e^{-5}$ (3e)	0.7139
+ Random seed = 2025 $\rightarrow$ 128 (3e)	0.7192
+ Weight decay = 0.01 (3e)	0.7065
`lert(hfl/chinese-lert-base)`	
3 epochs (default)	0.7022
+ Learning rate = $2e^{-5}$ (3e)	0.7122
+ Random seed = 2025 $\rightarrow$ 128 (3e)	0.7185
+ Weight decay = 0.01 (3e)	0.7049
`lert(hfl/chinese-lert-large)`	
3 epochs (default)	0.0003
+ Learning rate = $2e^{-5}$ (3e)	0.0003
+ Random seed = 2025 $\rightarrow$ 128 (3e)	0.7142
+ Weight decay = 0.01 (3e)	0.0000
`ELECTRA (hfl/chinese-electra-base-discriminator)`	
3 epochs (default)	0.7112
+ Learning rate = $2e^{-5}$ (3e)	0.7065
+ Random seed = 2025 $\rightarrow$ 128 (3e)	0.7115
+ Weight decay = 0.01 (3e)	0.7009

Table 4: Validation accuracy for QA-task where a prediction is counted correct if it is overlapping any part of the golden answers, for different hyperparameter settings of MacBERT, lert and ELECTRA.

Among all models, *ELECTRA*-base achieves the highest stable performance (0.7112), while *MacBERT-large* shows strong gains with adjusted learning rate and seed (up to 0.7192). *LERT-base* performs competitively, reaching 0.7185, whereas *LERT-large* suffers from instability, only improving under seed tuning (0.7142). Overall, careful hyperparameter selection notably enhances results.

Based on the experimental results presented in Table 4, I choose *LERT-base*. But having discussed with one classmate, who obtained around 0.79 in public test data with slightly difference configuration with `hfl/chinese-roberta-wwm-ext`. So in the end, I chose *LERT-base* and `hfl/chinese-roberta-wwm-ext`.

Apart from that, one of the significant findings is the consistent positive

impact of reducing the learning rate to 2×10^{-5} across all tested models. This adjustment resulted in performance gains ranging from 0.03 to 0.05 in validation accuracy. Other hyperparameter modifications showed model-specific effects with generally smaller improvements. Still, despite expecting performance of large variants, it's shown the underwhelming performance of large ones. And due to the instability, I will take `warm-up` to hyperparameter configuration.